

SUBPROGRAMS RECAP

1. Create a **procedure** called a **display_emp** in which you declare a cursor to select the name, position and date of employment of employees from the **Employees** table. Go through each line of the cursor and, if the date of employment is later than 01-AUG-1999, display the information retrieved. Call the procedure.

```
set serveroutput on;
CREATE OR REPLACE PROCEDURE display_emp IS
  CURSOR emp_cursor IS SELECT first_name || ' ' || last_name AS
    full_name, job_id, hire_date FROM employees;
  emp_record emp_cursor%ROWTYPE;

BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO emp_record;
    EXIT WHEN emp_cursor%NOTFOUND;
    IF emp_record.hire_date > TO_DATE('01-AUG-1999', 'DD-MON-YYYY') THEN
      DBMS_OUTPUT.PUT_LINE('Name: ' || emp_record.full_name);
      DBMS_OUTPUT.PUT_LINE('Position: ' || emp_record.job_id);
      DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(emp_record.hire_date, 'DD-
MON-YYYY'));
      DBMS_OUTPUT.PUT_LINE('-----');
    END IF;
  END LOOP;
  CLOSE emp_cursor;
END;
/
```

--Calling the procedure

```
BEGIN
  display_emp;
END;
/
```

2. Create a function called **seniority_employee (p_code employees.employee_id%type)** that returns the employee's seniority (calculated as the difference between the current date and the date of employment in months) that has the received code as a parameter. Handle exceptions. Call the function from a PL/SQL block and use a cursor to cycle through all employees.

```

CREATE OR REPLACE FUNCTION seniority_employee(p_code
EMPLOYEES.EMPLOYEE_ID%TYPE)
RETURN NUMBER
IS
CURSOR emp_cursor IS SELECT employee_id, hire_date FROM employees;
v_emp emp_cursor%ROWTYPE;
v_seniority NUMBER;
BEGIN
FOR v_emp IN emp_cursor LOOP
IF v_emp.employee_id = p_code THEN
v_seniority := FLOOR(MONTHS_BETWEEN(SYSDATE, v_emp.hire_date));
RETURN v_seniority;
END IF;
END LOOP;
RAISE_APPLICATION_ERROR(-20001, 'Employee ID ' || p_code || ' not found.');
```

```

EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
RETURN NULL;
END;
/
```

--Calling the function

```

DECLARE
v_id EMPLOYEES.EMPLOYEE_ID%TYPE := 101; -- You can test with any valid ID
v_days NUMBER;
BEGIN
v_days := seniority_employee(v_id);
IF v_days IS NOT NULL THEN
DBMS_OUTPUT.PUT_LINE('Employee ID ' || v_id || ' has ' || v_days || ' months in the
company.');
```

```

END IF;
END;
/
```

3. Create a procedure called **seniority_employee_proc (p_code IN employees.employee_id %type, p_seniority OUT number)** that calculates the seniority of the employee who has the code received as a parameter. Handle exceptions. Call the procedure from a PL/SQL block and use a cursor to cycle through all employees.

```

CREATE OR REPLACE PROCEDURE seniority_employee_proc(
p_code IN EMPLOYEES.EMPLOYEE_ID%TYPE,
p_seniority OUT NUMBER
)
IS
CURSOR emp_cursor IS SELECT employee_id, hire_date FROM employees;
```

```

v_emp emp_cursor%ROWTYPE;
BEGIN
p_seniority := NULL;

FOR v_emp IN emp_cursor LOOP IF v_emp.employee_id = p_code THEN
p_seniority := FLOOR(MONTHS_BETWEEN(SYSDATE, v_emp.hire_date) / 12);
RETURN;
END IF;
END LOOP;
RAISE_APPLICATION_ERROR(-20001, 'Employee ID ' || p_code || ' not found. ');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
        p_seniority := NULL;
END;
/
--Calling the procedure
DECLARE
v_emp_id EMPLOYEES.EMPLOYEE_ID%TYPE := 106;
v_seniority NUMBER;
BEGIN
seniority_employee_proc(v_emp_id, v_seniority);
IF v_seniority IS NOT NULL THEN
DBMS_OUTPUT.PUT_LINE('Employee ID ' || v_emp_id || ' has ' || v_seniority || ' years of
activity. ');
END IF;
END;
/

```

4. Create a procedure to return the end date and value of the most recent order: **info_order** (**p_date OUT orders.order_date%type, p_value OUT number**). Call the procedure from a PL/SQL block.

```

CREATE OR REPLACE PROCEDURE info_order(
    p_date OUT ORDERS.ORDER_DATE%TYPE,
    p_value OUT NUMBER
)
IS
v_order_id ORDERS.ORDER_ID%TYPE;
BEGIN
SELECT order_id, order_date INTO v_order_id, p_date FROM orders
WHERE order_date = (SELECT MAX(order_date)FROM orders);

SELECT SUM(unit_price * quantity) INTO p_value FROM order_items WHERE order_id =
v_order_id;

EXCEPTION
    WHEN NO_DATA_FOUND THEN

```

```

DBMS_OUTPUT.PUT_LINE('No orders found. ');
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
--Calling the procedure
DECLARE
v_date ORDERS.ORDER_DATE%TYPE;
v_value NUMBER;
BEGIN
info_order(v_date, v_value);
DBMS_OUTPUT.PUT_LINE('Most recent order date: ' || TO_CHAR(v_date, 'DD-MM-YY'));
DBMS_OUTPUT.PUT_LINE('Total value: ' || v_value);
END;
/

```

5. Create a trigger **positive_quantity** that does not allow negative values of the quantity that can be ordered in order_items table.

```

CREATE OR REPLACE TRIGGER positive_quantity
BEFORE INSERT OR UPDATE ON order_items
FOR EACH ROW
BEGIN
IF :NEW.quantity <= 0 THEN
RAISE_APPLICATION_ERROR(-20001, 'Quantity must be a positive value. ');
END IF;
END;
/

```

--Trigger fired example

```

INSERT INTO order_items (order_id, line_item_id, product_id, unit_price, discount_price,
quantity)
VALUES (9999, 1, 1001, 50, 45, -3);

```

6. Create a package called **update_jobs** that contains:
 - a. A procedure *add_jobs* to add a new record to the jobs table. The information to be added is provided as the parameters of the procedure. It is dealt with if there is already a function with the code entered.
 - b. A function *adjust_jobs* that changes the name of a job. The code of the function for which the change is made and the new name of the function are the parameters of the procedure. It is dealt with in the case where the change does not occur due to the specification of a code that is not found in the table.
 - c. A private procedure *delete_job* to delete a function based on the code received as a parameter. Deal with the case where the provided code does not exist.

```

CREATE OR REPLACE PACKAGE update_jobs IS
  PROCEDURE add_jobs(
    p_job_id JOBS.JOB_ID%TYPE,
    p_job_title JOBS.JOB_TITLE%TYPE,
    p_min_salary JOBS.MIN_SALARY%TYPE,
    p_max_salary JOBS.MAX_SALARY%TYPE
  );

  FUNCTION adjust_jobs(
    p_job_id JOBS.JOB_ID%TYPE,
    p_new_title JOBS.JOB_TITLE%TYPE
  ) RETURN VARCHAR2;
END update_jobs;
/

```

```

CREATE OR REPLACE PACKAGE BODY update_jobs IS
  PROCEDURE add_jobs(
    p_job_id JOBS.JOB_ID%TYPE,
    p_job_title JOBS.JOB_TITLE%TYPE,
    p_min_salary JOBS.MIN_SALARY%TYPE,
    p_max_salary JOBS.MAX_SALARY%TYPE
  ) IS
    v_count NUMBER;
  BEGIN
    SELECT COUNT(*) INTO v_count
    FROM jobs
    WHERE job_id = p_job_id;

    IF v_count > 0 THEN
      DBMS_OUTPUT.PUT_LINE('Job with ID ' || p_job_id || ' already exists.');
```

```
ELSE
```

```
  INSERT INTO jobs(job_id, job_title, min_salary, max_salary)
  VALUES (p_job_id, p_job_title, p_min_salary, p_max_salary);
```

```
  DBMS_OUTPUT.PUT_LINE('Job added successfully.');
```

```
END IF;
```

```
EXCEPTION
```

```
  WHEN OTHERS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Error adding job: ' || SQLERRM);
```

```
END add_jobs;
```

```
FUNCTION adjust_jobs(
```

```
  p_job_id JOBS.JOB_ID%TYPE,
```

```
  p_new_title JOBS.JOB_TITLE%TYPE
```

```
) RETURN VARCHAR2 IS
```

```
BEGIN
```

```
UPDATE jobs
SET job_title = p_new_title
WHERE job_id = p_job_id;
```

```
IF SQL%ROWCOUNT = 0 THEN
    RETURN 'No job found with ID ' || p_job_id || '.';
ELSE
    RETURN 'Job title updated for ID ' || p_job_id || '.';
END IF;
```

```
EXCEPTION
    WHEN OTHERS THEN
        RETURN 'Error updating job title: ' || SQLERRM;
END adjust_jobs;
```

```
PROCEDURE delete_job(p_job_id JOBS.JOB_ID%TYPE) IS
BEGIN
    DELETE FROM jobs
    WHERE job_id = p_job_id;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No job found with ID ' || p_job_id || ' to delete.');
```

```
    ELSE
        DBMS_OUTPUT.PUT_LINE('Job with ID ' || p_job_id || ' deleted.');
```

```
    END IF;
```

```
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error deleting job: ' || SQLERRM);
END delete_job;
```

```
END update_jobs;
/
```

```
BEGIN
    update_jobs.add_jobs('DEV_PROG', 'Developer', 4000, 8000);
END;
/
```

```
DECLARE
    v_result VARCHAR2(100);
BEGIN
    v_result := update_jobs.adjust_jobs('DEV_PROG', 'Senior Developer');
    DBMS_OUTPUT.PUT_LINE(v_result);
END;
```

